



CS1004 – Object Oriented Programming

Lecture # 14
Monday, April 04, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

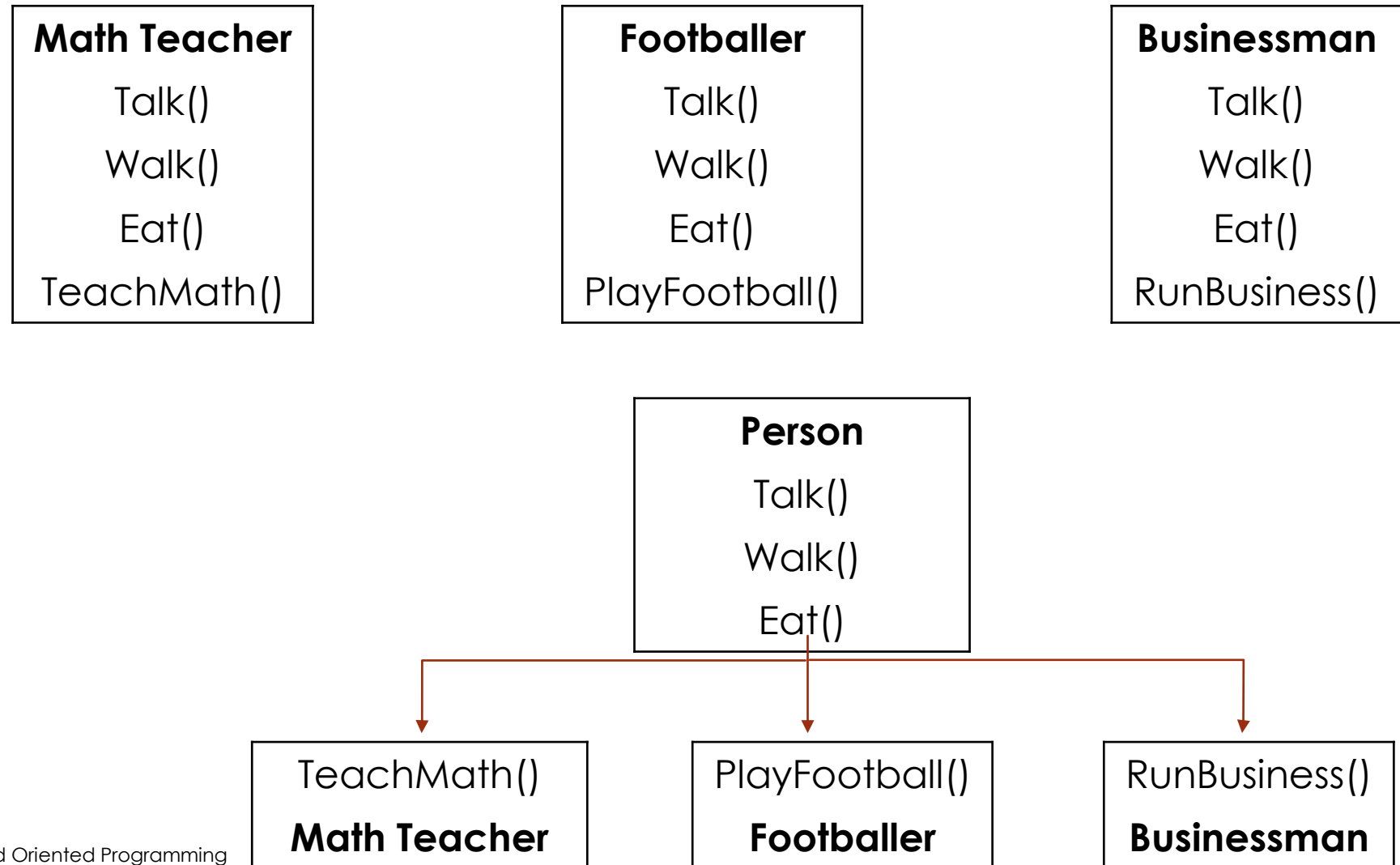
Muhammad Yousaf



2

Inheritance

Inheritance



Introduction

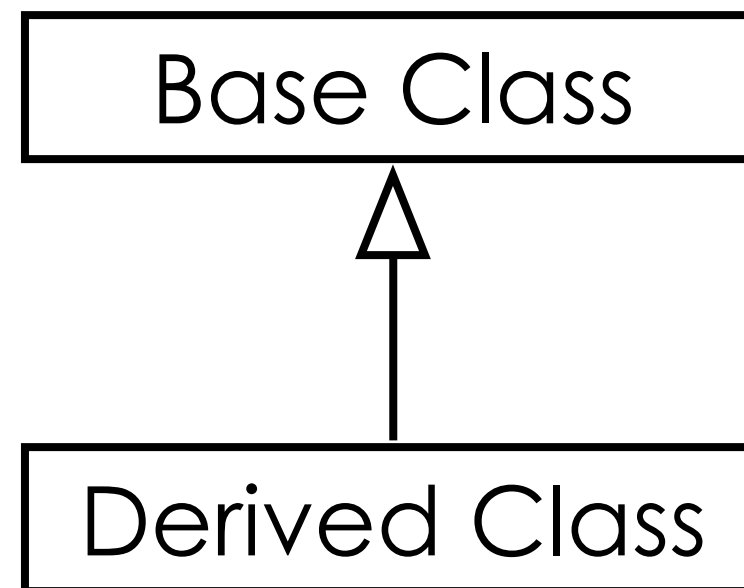
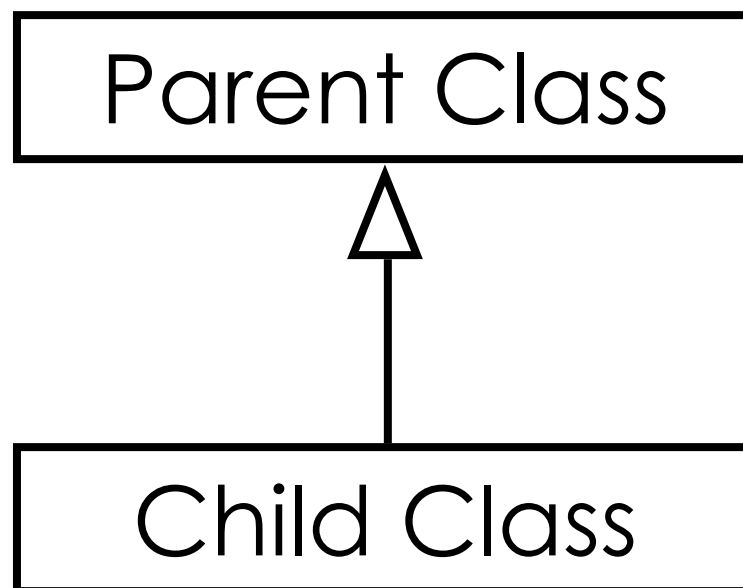
- Probably most powerful feature of OOP
- New classes are created from existing classes

Advantages

- Reusability
- Saves time and effort
- Improves program structure and reliability

Inheritance in classes

- ➡ If a class B inherits from class A, then B contains all the characteristics (information structure and behavior) of class A
- ➡ The parent class is called **base class** and the child class is called **derived class**
- ➡ Besides inherited characteristics, derived class may have its own unique characteristics



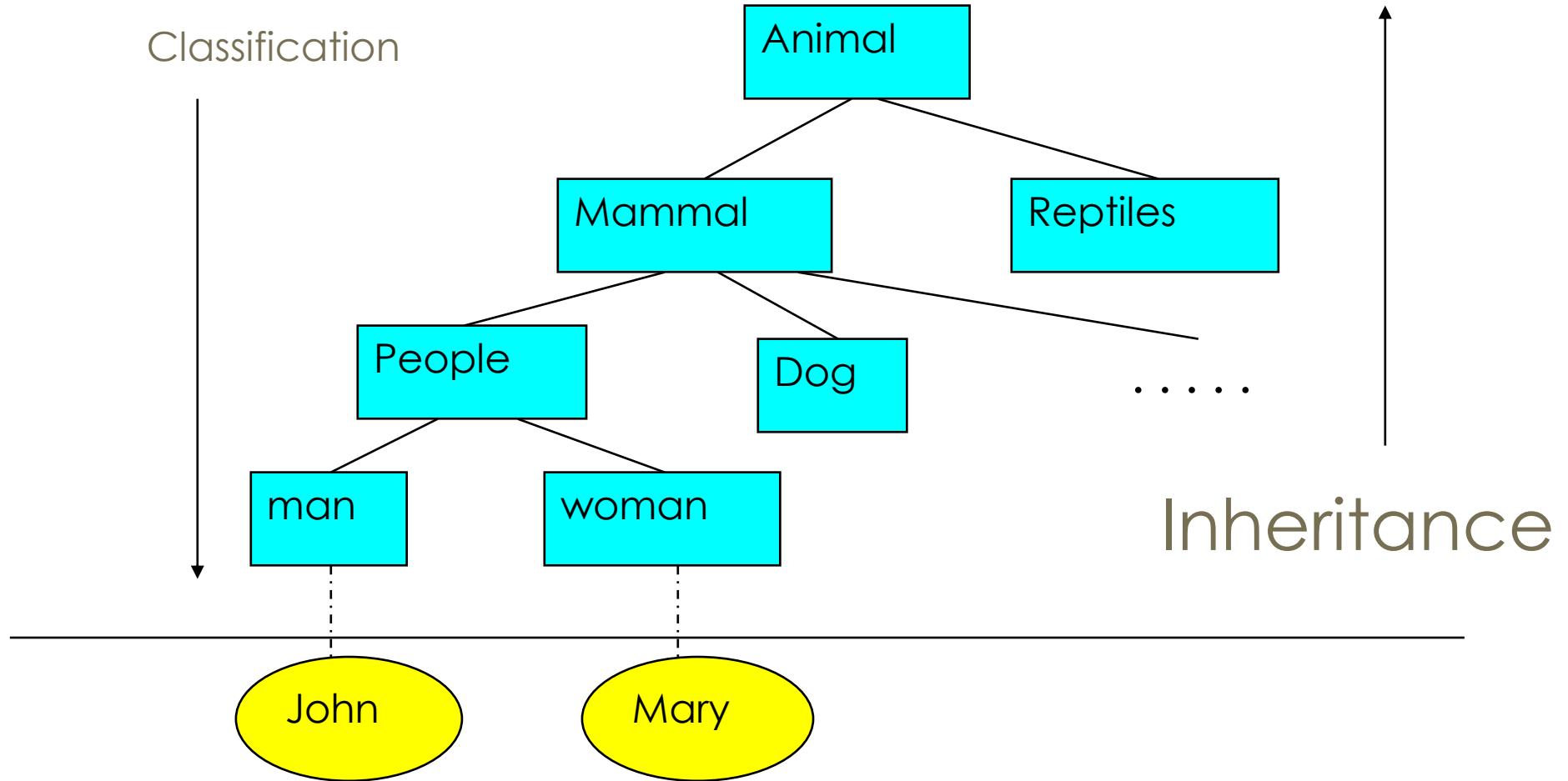
Inheriting Data and Functions

- All data members and member functions of base class are inherited to derived class
- Constructors, destructors and `= operator` are not inherited

Some definitions in class hierarchy

- Direct base class
 - Inherited explicitly (one level up hierarchy)
- Indirect base class
 - Inherited two or more levels up hierarchy
- Single inheritance
 - Inherits from one base class
- Multiple inheritance
 - Inheritance from multiple classes

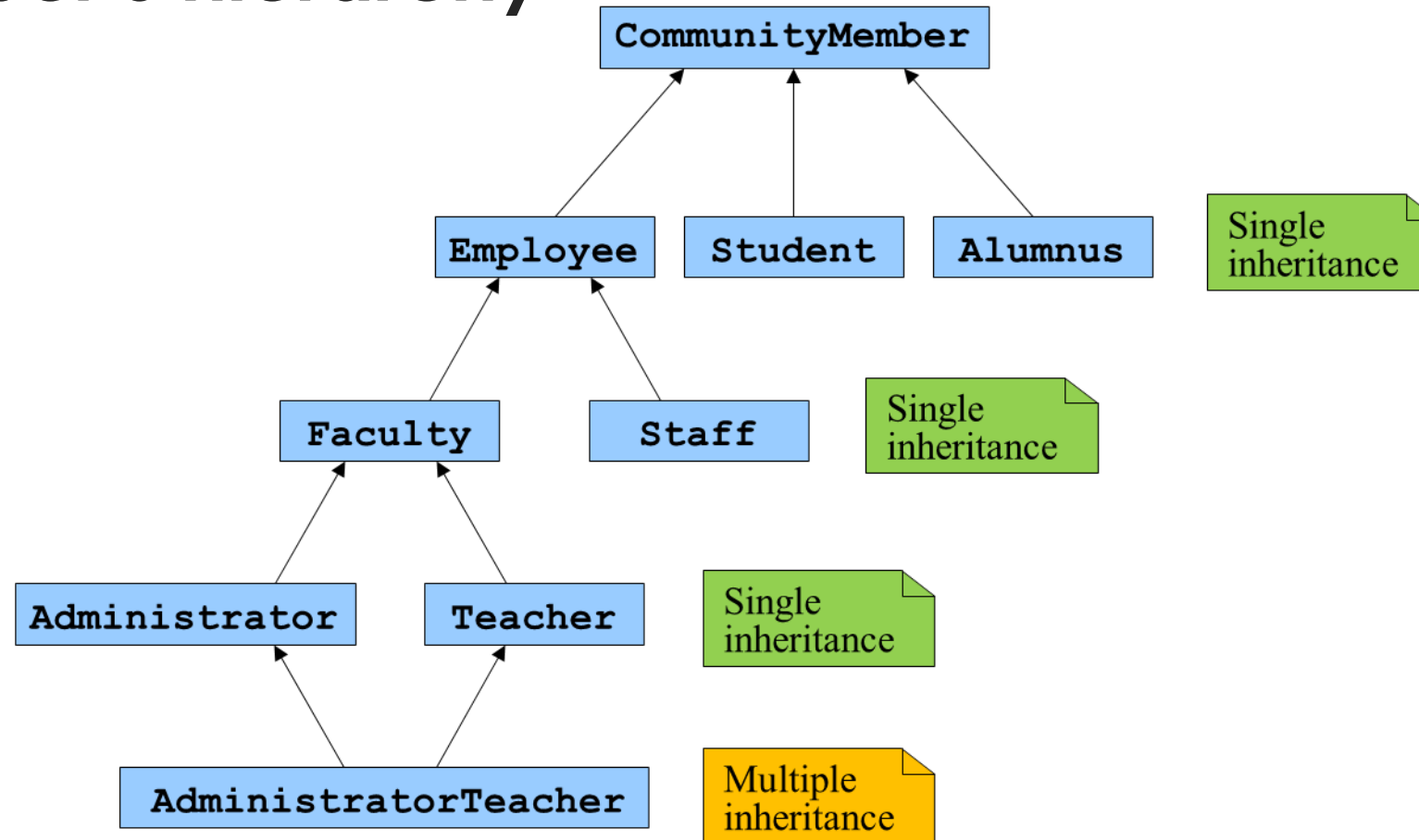
Animals: Class's hierarchy



Inheritance Examples

Base Class	Derived Class
Student	GraduateStudent UndergraduateStudent
Shape	Circle Triangle Rectangle
Loan	CarLoan HomeImprovementLoan
Employee	StaffMember FacultyMember
Account	CheckingAccount SavingAccount

Another example: University's community member's hierarchy



Inheritance in C++

- ➡ There are three types of inheritance in C++
 - ➡ Public
 - ➡ Private
 - ➡ Protected

Public Inheritance

- ➡ With **public inheritance**,
 - ➡ **public** and **protected** members of the base class become respectively **public** and **protected** members of the derived class

Protected Inheritance

➡ With **Protected inheritance**

➡ **Public** and **Protected** members of the base class become **Protected** members of the derived class

Private Inheritance

- ➡ With **private inheritance**
 - ➡ **Public** and **protected** members of the base class become **private** members of the derived class

Three modes of inheritance

```
class base
{
public:
    int x;
protected:
    int y;
private:
    int z;
};

class publicDerived : public base
{
    // x is public
    // y is protected
    // z is not accessible from
    //publicDerived
};
```

```
class protectedDerived : protected base
{
    // x is protected
    // y is protected
    // z is not accessible from
    //protectedDerived
};

class privateDerived : private base
{
    // x is private
    // y is private
    // z is not accessible from
    //privateDerived
}
```

Base class member access qualifier	Type of inheritance		
	Public inheritance	Protected inheritance	Private inheritance ⁺
Public	Public in derived class. Can be accessed directly by any non-static member function, friend function and non-member function	Protected in derived class. Can be accessed directly by any non-static member function and friend function	Private in derived class. Can be accessed directly by any non-static member function and friend function
Protected	Protected in derived class. Can be accessed directly by all non-static member functions and friend functions	Protected in derived class. Can be accessed directly by all non-static member functions and friend functions	Protected in derived class. Can be accessed directly by all non-static member functions and friend functions
Private	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of base class	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of base class	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of base class

Define a Class Hierarchy

➤ Syntax:

```
class DerivedClassName : access-level BaseClassName
```

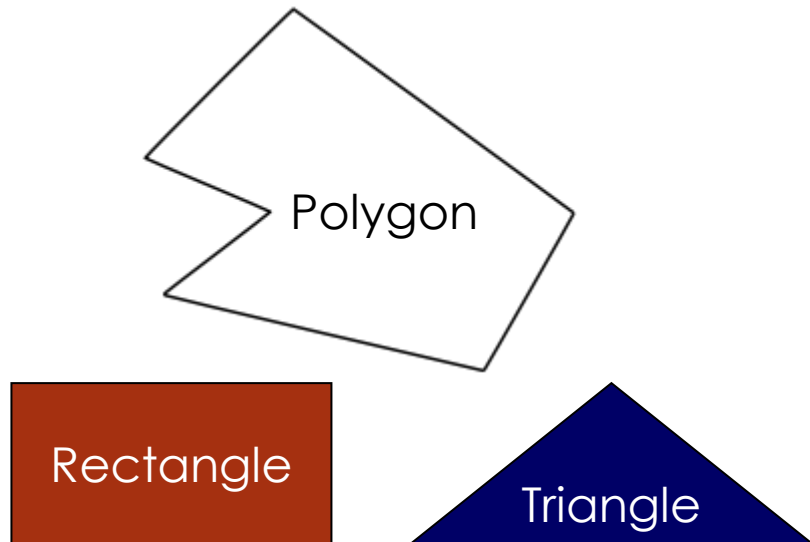
➤ Where

- access-level specifies the type of derivation
- private by default, protected or public

➤ Any class can serve as a base class

➤ Thus a derived class can also be a base class

Inheritance concept

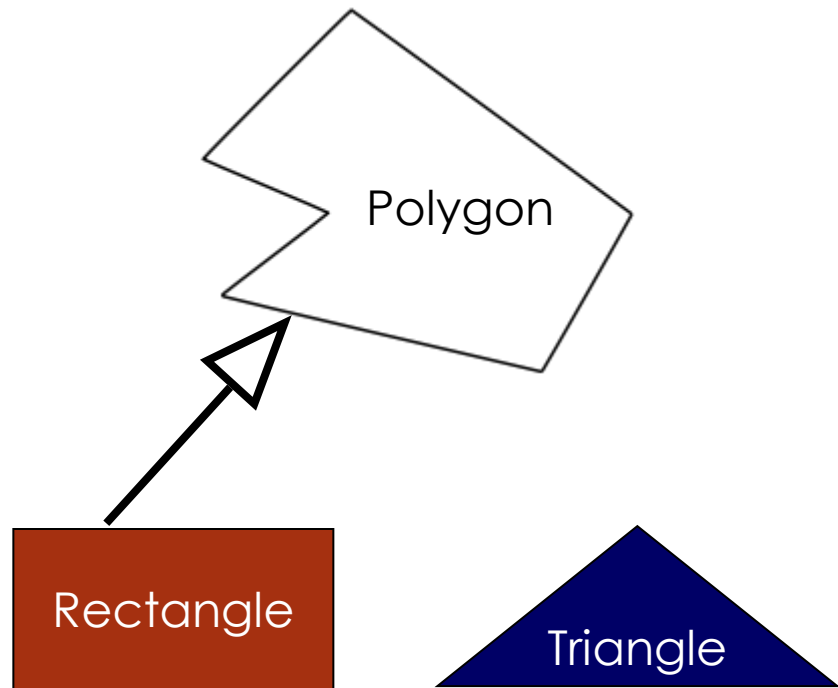


```
class Polygon {
private:
    int numVertices;
    float *xCoord, *yCoord;
public:
    void set(float *x, float *y, int nV);
};
```

```
class Rectangle {
private:
    int numVertices;
    float *xCoord, *yCoord;
public:
    void set(float *x, float *y, int nV);
    float area();
};
```

```
class Triangle {
private:
    int numVertices;
    float *xCoord, *yCoord;
public:
    void set(float *x, float *y, int nV);
    float area();
};
```

Inheritance concept



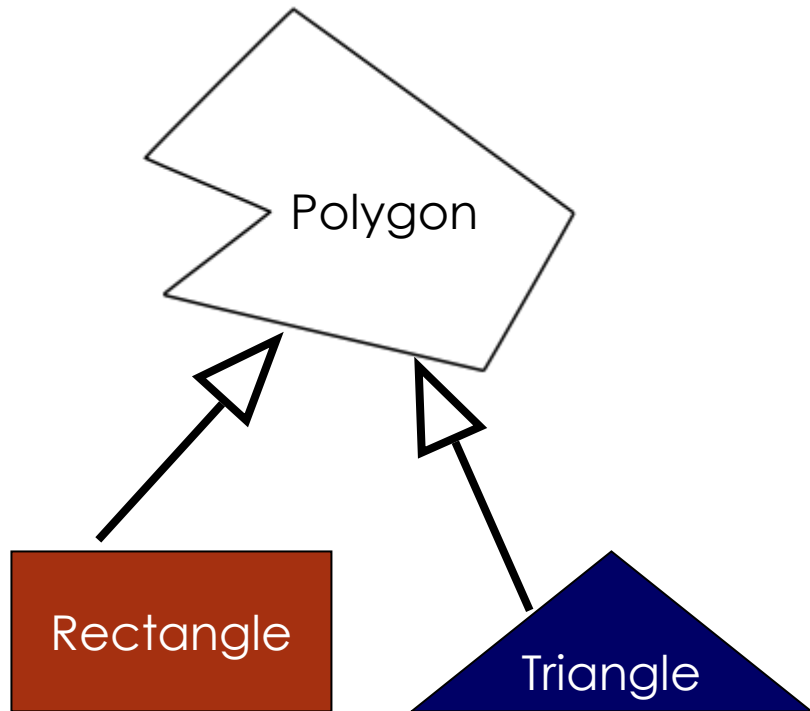
```
class Rectangle: public Polygon {  
public:  
    float area();  
};
```

```
class Polygon {  
protected:  
    int numVertices;  
    float *xCoord, *yCoord;  
public:  
    void set(float *x, float *y, int nV);  
};
```

```
class Rectangle {  
private:  
    int numVertices;  
    float *xCoord, *yCoord;  
public:  
    void set(float *x, float *y, int nV);  
public:  
    float area();  
};
```



Inheritance concept



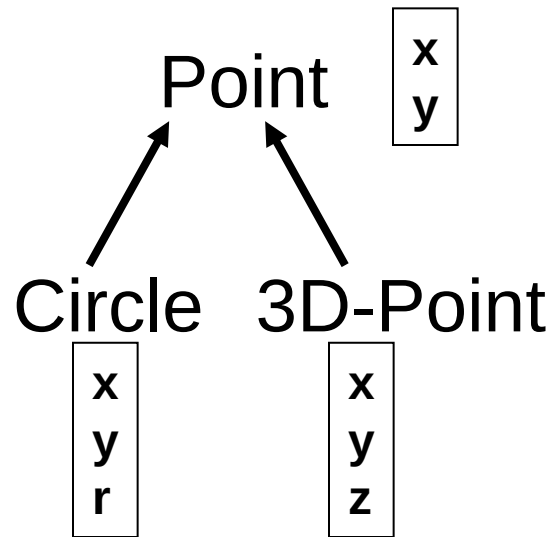
```
class Triangle: public Polygon {
public:
    float area();
};
```

```
class Polygon {
protected:
    int numVertices;
    float *xCoord, *yCoord;
public:
    void set(float *x, float *y, int nV);
};
```

```
class Triangle {
private:
    int numVertices;
    float *xCoord, *yCoord;
public:
    void set(float *x, float *y, int nV);
public:
    float area();
};
```



Inheritance concepts using point

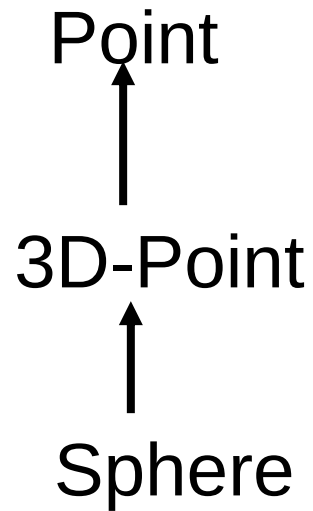


```
class Point{
    protected:
        int x, y;
    public:
        void set (int a, int b);
};
```

```
class Circle : public Point{
    private:
        double r;
};
```

```
class 3D-Point: public Point{
    private:
        int z;
};
```

Class derivation



```
class 3D-Point : public Point{  
    private:  
        double z;  
        ... ..  
};
```

```
class Point{  
    protected:  
        int x, y;  
    public:  
        void set (int a, int b);  
};
```

```
class Sphere : public 3D-Point{  
    private:  
        double r;  
        ... ..  
};
```

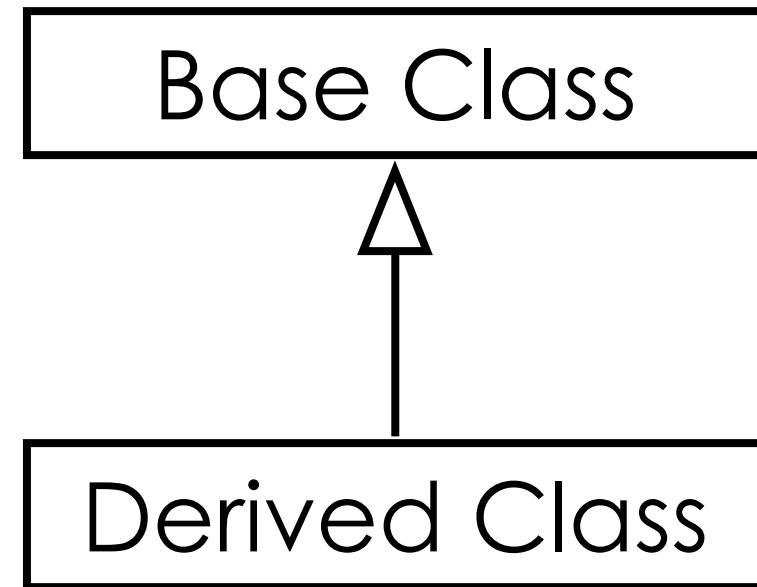
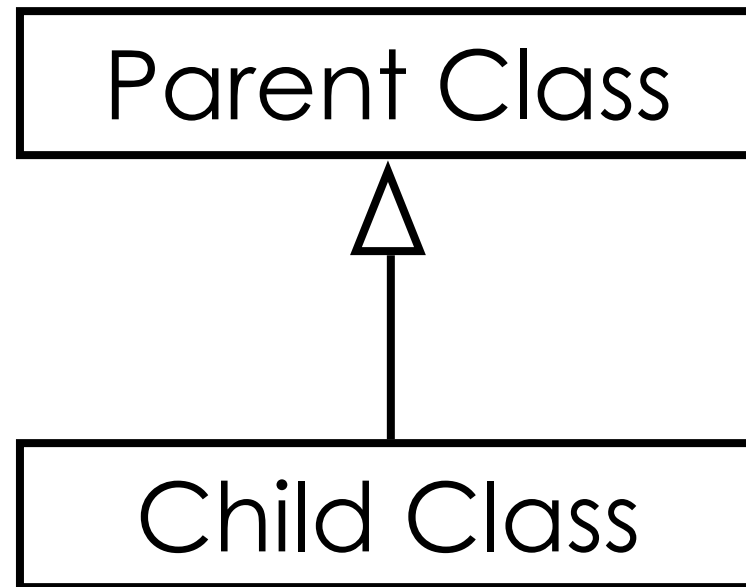
Point is the base class of 3D-Point, while 3D-Point is the base class of Sphere

What to inherit

- **In principle**, every member (but not private) of a base class is inherited by a derived class
 - just with different access permission

Task

- Write a class employee that contains attributes of employee id and scale. The class contains member functions to input and output these attributes. Write a child class Manager that inherits Employee class. The child class has attributes of manager name and his department and member function to show all attributes(employee id, scale, name, department)



“IS A” Relationship

➡ IS A relationship is modeled with the help of public inheritance

➡ **Syntax**

```
class ChildClass : public BaseClass {  
    ...  
};
```

Accessing Members

- ➡ Public members of base class become public member of derived class
- ➡ Private members of base class are not accessible from outside of base class, even in the derived class (Information Hiding)

Extending the Student example

```
class Person {  
    char *name;  
    int age;  
    ...  
public:  
    const char *GetName() const;  
    int GetAge() const;  
    void setName(const char*) ;  
    void setAge(const int);  
    ...  
};
```

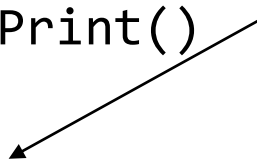
Inheriting Student class from Person

```
class Student : public Person {  
    int semester;  
    int rollNo;  
    ...  
public:  
    int GetSemester() const;  
    int GetRollNo() const;  
    void Print() const;  
    ...  
};
```

Accessing member of base class

```
void Student::Print()  
{  
    cout << name << " is in" << " semester " << semester;  
}
```

It will generate an error as name is a private member of base class



Accessing member of base class

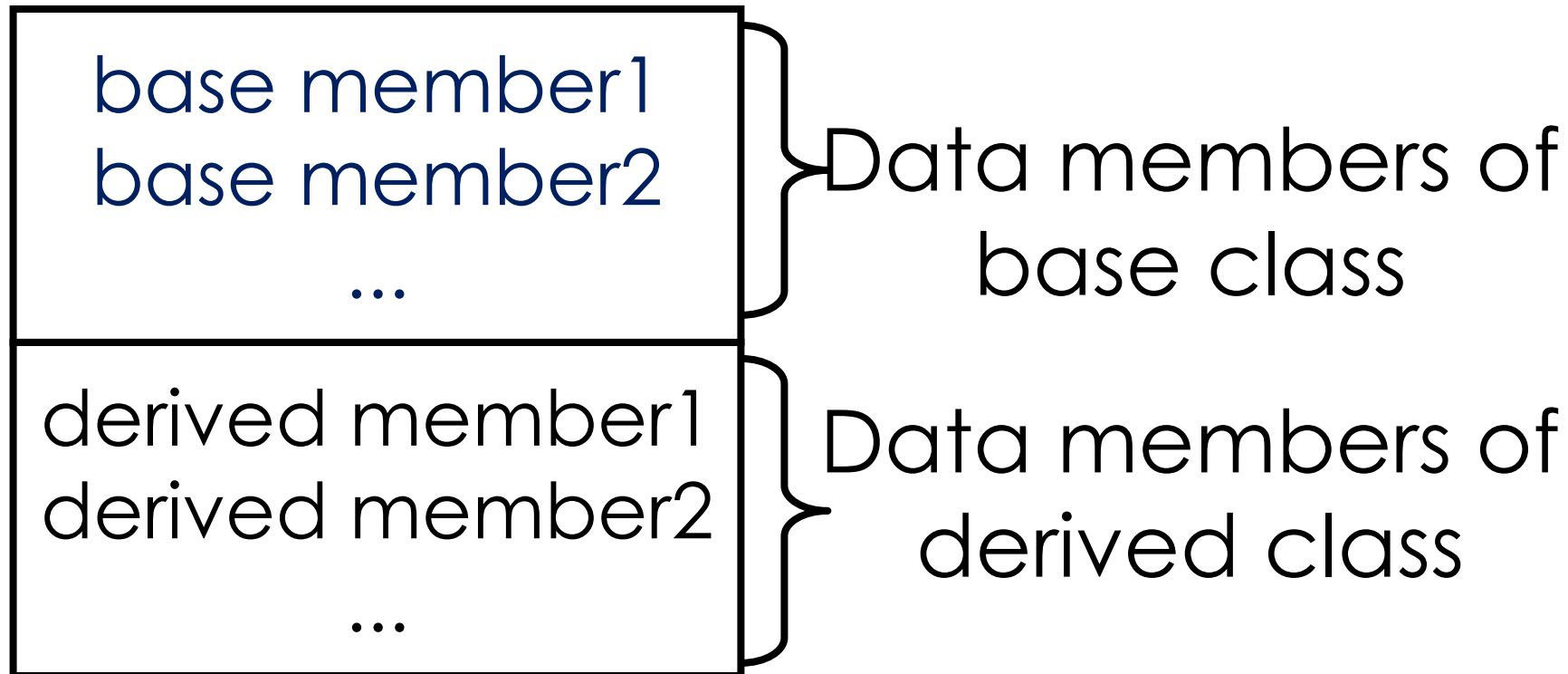
```
void Student::Print()  
{  
    cout << GetName() << " is in semester " << semester;  
}
```

Driver program to test Student class

```
int main() {  
    Student stdt;  
  
    stdt.semester = 0; //error  
    stdt.name = NULL; //error  
    cout << stdt.GetSemester();  
    cout << stdt.GetName();  
    return 0;  
}
```

Memory footprint

- The object of derived class is represented in memory as follows



Memory footprint

- ➡ Every object of derived class has an anonymous object of base class

Constructors

- The anonymous object of base class must be initialized using constructor of base class
- When a derived class object is created the constructor of base class is executed before the constructor of derived class

Constructors

base member1
base member2
...

Base class constructor
initializes the anonymous
object

derived member1
derived member2
...

Derived class constructor
initializes the derived class
object

Constructors of base and child class

```
class Parent {  
public:  
    Parent() {  
        cout << "Parent Constructor...";  
    }  
};  
class Child : public Parent {  
public:  
    Child() {  
        cout << "Child Constructor...";  
    }  
};
```

Constructors of base and child class

```
int main() {  
    Child cobj;  
    return 0;  
}
```

Output:

Parent Constructor...

Child Constructor...

Constructor

- ➡ If default constructor of base class does not exist, then the compiler will try to generate a default constructor for base class and execute it before executing constructor of derived class
- ➡ If the user has given only an overloaded constructor for base class, the compiler will not generate default constructor for base class

Constructor

```
class Parent {  
    public:  
        Parent(int i) {}  
};
```

```
class Child : public Parent {  
    public:  
        Child() {}  
} Child_Object; //ERROR
```

Base Class Initializer

- ➡ C++ has provided a mechanism to explicitly call a constructor of base class from derived class
- ➡ The syntax is similar to member initializer and is referred as base-class initialization

Base class initializer

```
class Parent {  
    public:  
        Parent(int i) { ... }  
};
```

```
class Child : public Parent {  
    public:  
        Child(int i) : Parent(i)  
        {  
            ...  
        }  
};
```

Example

```
class Parent {  
public:  
    Parent() {  
        cout << "Parent Constructor...";  
    }  
    ...  
};  
class Child : public Parent {  
public:  
    Child() :Parent()  
    {  
        cout << "Child Constructor...";  
    }  
    ...  
};
```

Base Class Initializer

- ➡ User can provide base class initializer and member initializer simultaneously

Base and Child class initializers simultaneously

```
class Parent {  
    public:  
        Parent() { ... }  
};  
  
class Child : public Parent {  
    int member;  
    public:  
        Child() :member(0), Parent()  
        {  
            ...  
        }  
};
```

Base Class Initializer

- ➡ The base class initializer can be written after member initializer for derived class
- ➡ The base class constructor is executed before the initialization of data members of derived class

Questions

